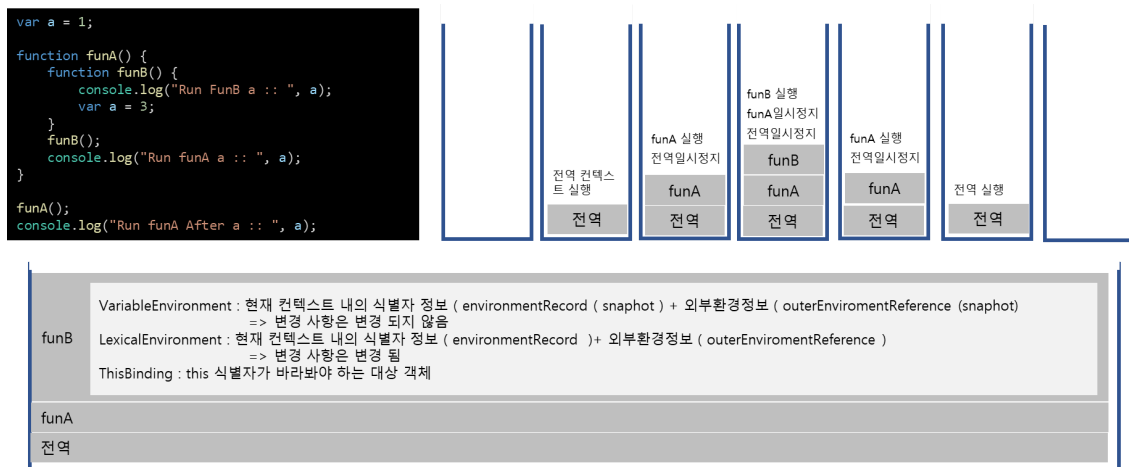


2. 변수 (스코프)

1. 실행 컨텍스트

- 문맥 (context) : 코드를 실행할때 코드
- 렉시컬 환경(lexical environment) : 스크립트 전체, 실행중인 함수, 코드블록 등은 자신만의 렉시컬 환경으로 환경레코드, 외부렉시컬 환경으로 구성됩니다.
 - 환경레코드 : 렉시컬 환경에서 모든 지역변수를 프로퍼티로 저장하고 있는 객체
 - 외부렉시컬 환경 : 현재 렉시컬 환경보다 더 상위의 렉시컬 환경으로. 스크립트는 최상위 렉시컬 환경이며 스크립트 내에 호출된 함수나 코드블록은 외부렉시컬 환경으로 스크립트 렉시컬 환경을 참조합니다.
 - 실행 컨텍스트(execution context) : 코드의 문맥(context)은 렉시컬 환경으로 이루어진다. 이를 구현한 것으로 모든 컨텍스트에서 평가되고 실행됩니다.



실행 컨텍스트

2. 스코프

스코프는 모든 식별자(변수 이름, 함수 이름, 클래스 이름 등)는 자신이 선언된 위치에 의해 다른 코드가 식별자 자신을 참조할 수 있는 유효 범위가 결정하게 되는 것을 의미 합니다.

- 변수와 매개변수의 접근성과 생존 시간을 의미 함
- 전역범위(Global Scope)와 지역 유효 범위(Local Scope)로 나뉜다.
- 전역범위는 전체를 의미 하며 지역 범위는 정의 된 함수 내에서만 참조 되는 것을 의미 함 (Function-Level Scope)
- 다른 언어는 Block-level Scope 를 사용 하는데 ES6에 등장한 let, const를 사용 하면 block-level scope를 사용할 수 있다.

2-1. 스코프의 종류

- 전역 :
 - 어디서든지 참조할 수 있습니다
 - 생명 주기는 애플리케이션의 생명 주기와 같습니다.
- 지역 :
 - 블록 내부에서 만 참조할 수 있습니다.
 - 생명 주기는 함수의 생명 주기와 일치합니다.

```

var gVal = "global"; // 전역 Scope

function fFuncLevel01() {
  var fLevel01Val = "1 Level";
  console.log("gVal ::", gVal); // gVal :: global
  function fFuncLevel02() {
    console.log("fLevel01Val ::", fLevel01Val); // 1 Level
    var fLevel02Val = "2 Level";
    console.log("fLevel02Val ::", fLevel02Val); // 2 Level
  }
  fFuncLevel02();
  //console.log("fLevel02Val ::", fLevel02Val);
  // => ReferenceError: fLevel02Val is not defined
}

fFuncLevel01();
// console.log("fLevel01Val ::", fLevel01Val);
// => ReferenceError: fLevel02Val is not defined

```

비 블록 레벨 스코프(Non block-level scope)

```

function fFunc() {
  var a = 0;
  if (true) {
    var b = 0; // 비 블록 레벨 스코프(Non block-level scope)
    for ( var c = 0; c < 5; c++) {
      // 작업
    }
    console.log("c=",c); // c= 5
  }

  console.log("b=",b); // b= 0
}
fFunc();

```

2-2. 스코프 체인

일종의 리스트로서 전역 객체와 중첩된 함수의 스코프의 레퍼런스를 차례로 저장하고, 의미 그대로 각각의 스코프가 어떻게 연결(chain)되고 있는지 보여주는 것을 의미합니다. 즉 변수를 참조할 때 자바스크립트 엔진은 스코프 체인을 통해 변수를 참조하는 코드의 스코프에서 시작하여 상위 스코프 방향으로 이동하여 선언된 변수를 검색합니다. 따라서, **상위 스코프에서 유효한 함수는 하위 스코프에서 자유롭게 참조가 가능하지만, 하위 스코프에서 유효한 변수를 상위 스코프에서는 참조할 수 없습니다.**

- 함수 외부에 선언한 변수는 함수 내부에서도 사용 가능 하지만 함수 내부에 선언한 변수는 함수 내부에서만 가능 하다.
- Scpoe Chain은 식별자의 유효 범위를 함수 내부에서 외부로 차례로 검색 하는 것이다.

```

var a = 1;
var funA = function() {

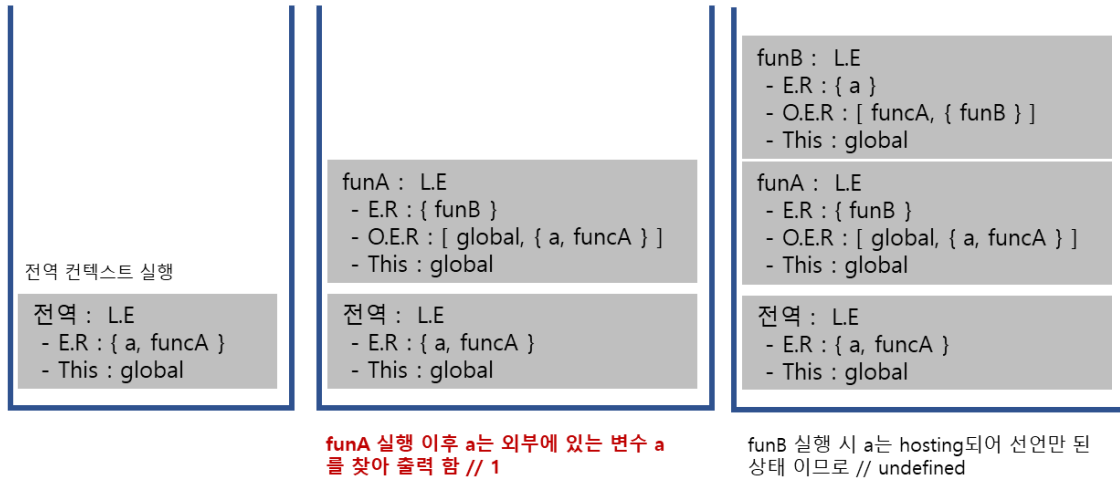
```

```

var funB = function() {
  console.log(a); // undefined
  var a = 2; // 변수 은닉화
};
funB();
console.log(a); // 1
}

funA();
console.log(a); // 1

```



2-3. 함수 레벨 스코프

코드 블록이 아닌 함수에 의해서만 지역 스코프가 생성된다는 것으로 `var` 키워드로 선언된 변수는 오로지 함수의 코드 블록(함수 몸체)만을 지역 스코프로 인정한다.

```

> var i = 10;
  for ( var i = 0 ; i < 5 ; i++ ) {
    console.log(` for 블록 내부 i : ${i} `);
  }
  console.log(` 전역 i : ${i} `);

```

```

for 블록 내부 i : 0
for 블록 내부 i : 1
for 블록 내부 i : 2
for 블록 내부 i : 3
for 블록 내부 i : 4
전역 i : 5

```

2-4. 렉시컬 스코프

자바스크립트는 렉시컬 스코프로 함수의 호출위치는 상위 스코프 결정에 어떠한 영향도 주지 않는다. 즉 함수의 상위 스코프는 언제나 자신이 정의된 스코프입니다.

함수의 상위 스코프는 언제나 자신이 정의된 스코프 이다.

- **동적 스코프(dynamic scope)** : 함수를 어디서 호출했는지에 따라 함수의 상위 스코프를 결정

- **렉시컬 스코프(lexical Scope)** : 함수를 어디서 정의했는지에 따라 함수의 상위 스코프를 결정

즉 렉시컬 스코프는 함수를 어디서 호출하는지가 아니라 어디에 선언하였는지에 따라 결정된다

```
> var strName = "홍";

//정의
const fnName00 = () => {
  var strName = "김";
  console.log(`fnFiestName 내부 strName : ${strName} 길동`);
  fnName01();
}

//정의
const fnName01 = () => {
  console.log(`fnFiestName 전역 strName : ${strName} 길동`);
}

fnName00();
fnName01();
```

fnFiestName 내부 strName : 김 길동
fnFiestName 전역 strName : 홍 길동
fnFiestName 전역 strName : 홍 길동

```
function f1(){
  var a = 10;  f2();
}
function f2(){
  console.log("호출 ");
}
f1();  // 결과 10

function f1(){
  a = 10;
  console.log(f2());
}

function f2(){
  return a;
}

f1();  // 결과 10

var x = 1;
function foo() {  // 전역에 선언
  var x = 10;
  bar();
}

function bar() {  // 전역에 선언
  console.log(x);
}

foo();  // 결과 : 1
```

```
bar(); // 결과 : 1
```

3. 변수

자바스크립트에서 변수 선언은 `var`를 사용하는 것 밖에 없었으나 이제는 `var`, `let`, `const`을 사용할 수 있으며 변수는 자신이 선언된 위치(전역 혹은 지역)에 의해 자신의 유효 범위인 스코프가 결정됩니다.

3-1. var

함수 스코프(Function Scope)만 지원 하고 블록 스코프(Block Scope)를 지원하지 않으므로 블록 내부에서 사용한 `var` 변수 선언하여도 블록 외부에서 사용할 수 있습니다.

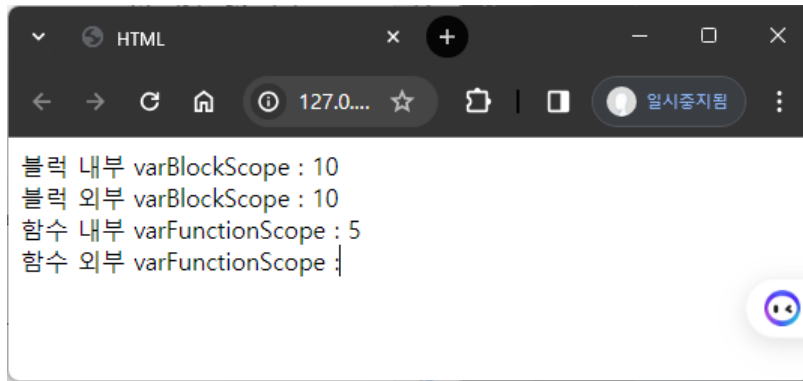
```
{% code lineNumbers="true" %}
```

```
<body>
  블록 내부 varBlockScope : <Label id="blockInner" ></Label></br>
  블록 외부 varBlockScope : <Label id="blockOuter" ></Label></br>
  함수 내부 varFunctionScope : <Label id="functionInner" ></Label></br>
  함수 외부 varFunctionScope :<Label id="functionOuter" ></Label></br>
</body>
<script>
  var varBlockScope = 5;
  {
    // var 선언 변수는 블록 범위가 아니므로 재선언할 수 있음
    var varBlockScope = 10 ;
    document.querySelector("#blockInner").innerHTML = varBlockScope;
  }
  document.querySelector("#blockOuter").innerHTML = varBlockScope;

  fnVar();
  function fnVar() {
    var varFunctionScope = 5;
    document.querySelector("#functionInner").innerHTML =
varFunctionScope;
  }
  // Function 으로 오류 발생
  document.querySelector("#functionOuter").innerHTML = varFunctionScope;
</script>
```

```
{% endcode %}
```

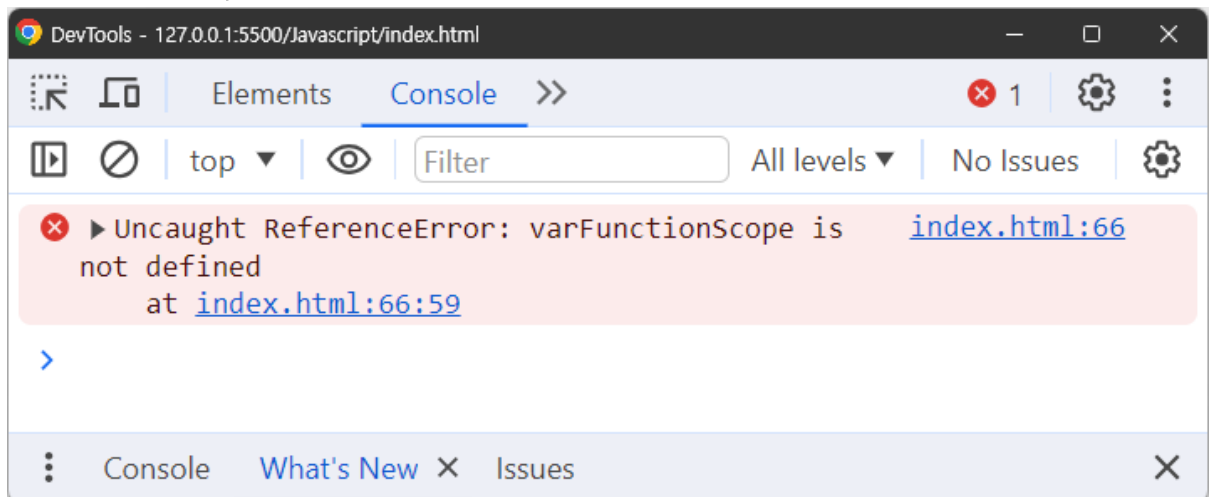
- `varBlockScope` 변수는 8라인, 11 라인에서 `var` 지시자로 선언 및 할당 되었습니다. 9라인에서 블록 내부에 재선언되어 있으나 실제 `varBlockScope` 변수의 유효 범위는 8라인에서 선언한 유효 범위를 적용받아 `#blockInner`, `#blockOuter` 에 모두 10으로 표시 됩니다.
- `varFunctionScope` 변수는 `fnVar` 함수 내부에 선언이 되어 유효 범위는 `fnVar` 함수 내부에서만 유효하므로 `#functionOuter`에서는 오류가 발생 합니다.



수행 결과

- 소스 설명

- 9 라인 : var 으로 varBlockScope 변수를 선언 하고 5값으로 초기화
- 전체 범위에 적용이 됩니다.
- 11 라인 : varBlockScope 변수를 블록 범위에서 10 값으로 설정
- var으로 varBlockScope 변수를 선언하였지만 var는 블록 스코프에 적용이 되지 않으므로 실제 9라인에서 선언한 변수에 10 값을 설정 함
- 12 라인 : id가 blockInner인 Label에 표시 : 10으로 표시됨
- 14 라인 : id가 blockOuter인 Label에 표시 : 10으로 표시됨
- 5로 표시되지 않는 이유는 varBlockScope는 var로 선언되어 함수 스코프만 지원 되므로 11라인에서 설정한 값을 가지고 있기 때문입니다.
- 19 라인 : var으로 varFunctionScope를 변수를 선언하고 5로 초기화
- 23 라인 : varFunctionScope 값을 id가 functionOuter에 표시 하라고 하였지만 오류 발생
- varFunctionScope 변수는 함수 스코프로 외부에서 사용 할 수 없습니다.



3-1-1. var 키워드 특징

- 블록 스코프를 지원하지 않는다
- 같은 이름으로 중복 선언할 수 있습니다. (실제 : 할당으로 보아야 함)

3-1-2. var 키워드 생략 (암묵적 전역)

```
function fFunc() {
  gVal = "local";
}
```

```
function fFunc01() {
    console.log("gVal =", gVal); // local
}
fFunc();
fFunc01();
```

3-1-3. 변수명 중복

```
var gVal = "global"; function fFunc() {
    var gVal = "local";
    console.log("gVal =", gVal); // gVal = local
}

fFunc();
console.log("gVal =", gVal); // gVal
```

3-2. let

블록 스코프를 지원하는 변수 선언 키워드로 값을 변경할 수 있습니다.

- 중복 선언 불가능
- 호이스팅이 발생하지 않는다.
- for문 외부에서 사용 불가능

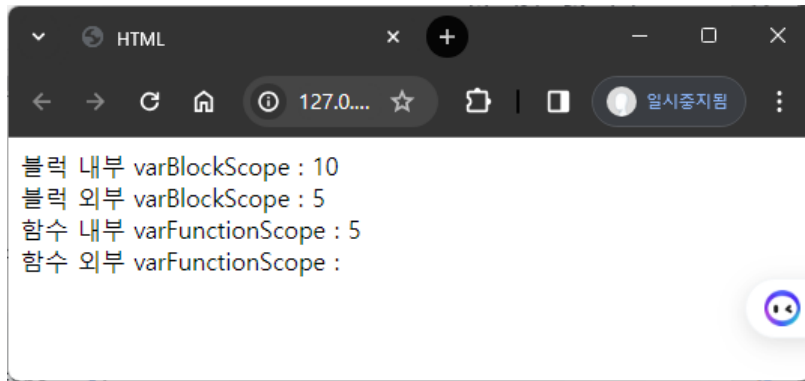
var 예제를 let으로만 변경한 코드 입니다.

{% code lineNumbers="true" %}

```
<script>
    let varBlockScope = 5;
    {
        let varBlockScope = 10 ;
        document.querySelector("#blockInner").innerHTML = varBlockScope;
    }
    document.querySelector("#blockOuter").innerHTML = varBlockScope;

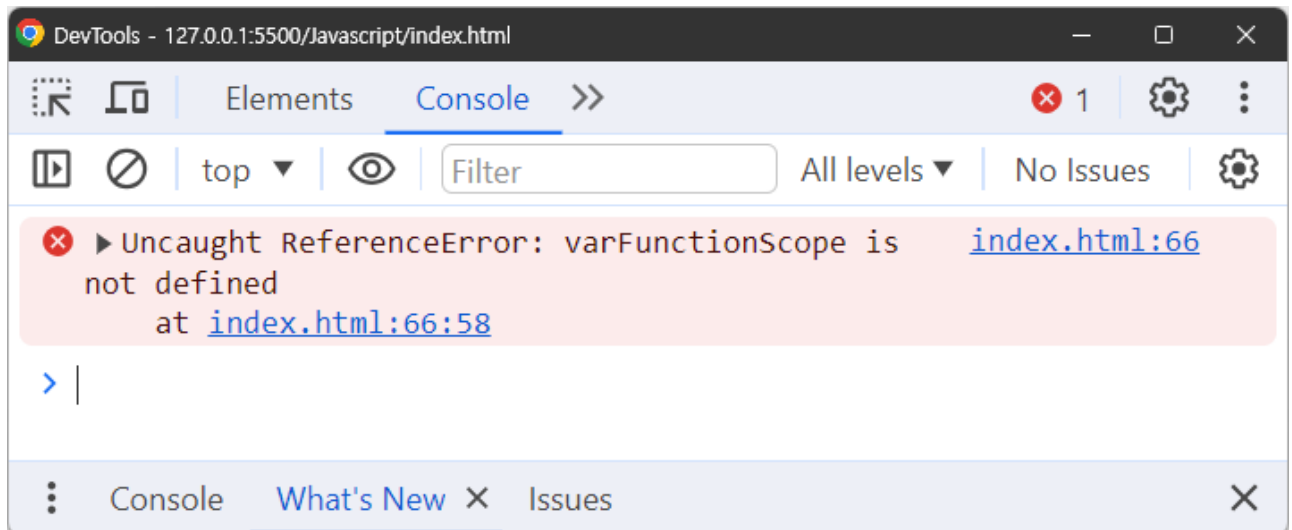
    fnVar();
    function fnVar() {
        let varFunctionScope = 5;
        document.querySelector("#functionInner").innerHTML =
varFunctionScope;
    }
    // varFunctionScope는 선언이 되어 있지 않아서 오류 발생
    document.querySelector("#functionOuter").innerHTML = varFunctionScope;
</script>
```

{% endcode %}



let 변수 선언으로 변경 한 결과

- 7 라인 : blockOuter Label에 5로 표시 되는 것이 확인 됨
- 2 라인에서 선언한 varBlockScope 변수에 영향을 받습니다. 4라인에서 동일 변수명 (varBlockScope)을 let으로 선언 하였으므로 2라인의 varBlockScope 변수는 7라인에서 선언한 변수가 영향을 줍니다.
- 15 라인 : varFunctionScope 변수는 선언이 되어 있지 않아서 오류가 발생 합니다.



3-3. const

let 키워드와 같게 블록 스코프를 지원하여 블록 내부로 제한을 합니다. let 키워드와 틀린 점은 한 번 값을 할당 하면 바꿀수 없습니다. 다시 말하면 재할당을 할 수 없다는 것 입니다. 즉 불변값이 되는 것은 아닙니다. 배열과 같은 오브젝트 객체의 값은 변경 할 수 있습니다.

- const 키워드로 선언한 변수는 선언과 동시에 초기화 해야한다.
- 재할당 금지

```
{% code lineNumbers="true" %}
```

```
<body>
  const 할당  constVariable : <Label id="constVar" ></Label></br>
  const 재 할당 constVariable : <Label id="changeConstVar" ></Label></br>
</body>
<script>
  {
    const constVariable = 10 ;
    document.querySelector("#constVar").innerHTML = constVariable;
    // constVariable 값 변경 ( 재 할당 ) -> 오류 발생
```

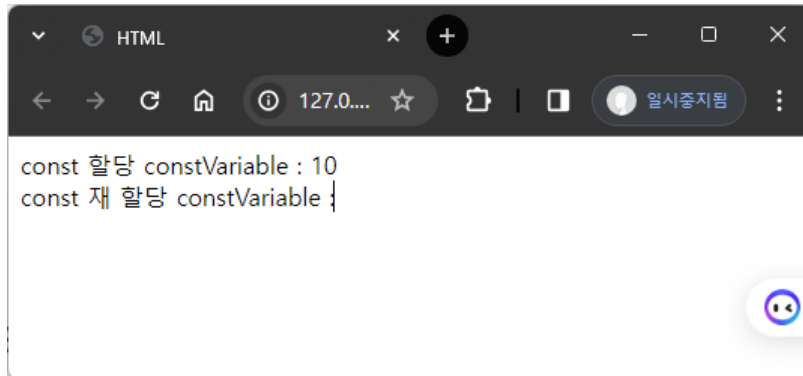


```

        constVariable = constVariable + 10;
        document.querySelector("#changeConstVar").innerHTML =
constVariable;
    }
</script>

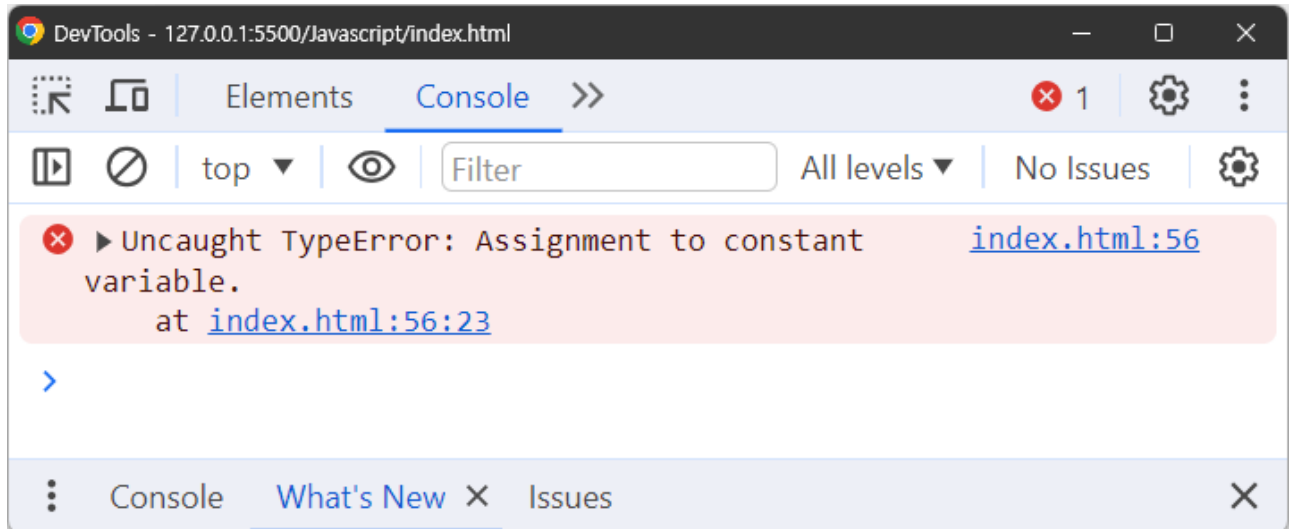
```

{% endcode %}



수행 결과

- 7 라인 : const 키워드로 constVariable 선언 및 초기화
- 10 라인 : const 키워드로 constVariable 변수 값 변경
 - const 키워드 선언한 변수는 재할당을 할 수 없어서 오류 발생



const 키워드로 선언한 변수가 객체를 값으로 가지고 있으면 값을 변경 할 수 있습니다. 즉 불변값은 아니라는 것 입니다.

```

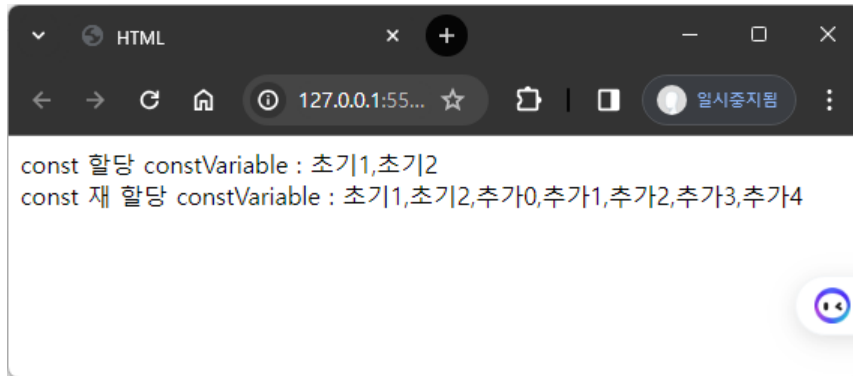
<body>
    const 할당  constVariable : <Label id="constVar" ></Label></br>
    const 재 할당 constVariable : <Label id="changeConstVar" ></Label></br>
</body>
<script>
{
    const constArray = ["초기1", "초기2"] ;
    document.querySelector("#constVar").innerHTML = constArray;
    for ( let i = 0; i < 5 ; i ++ ) {
        // 값을 변경 하고 있음
    }
}

```

```

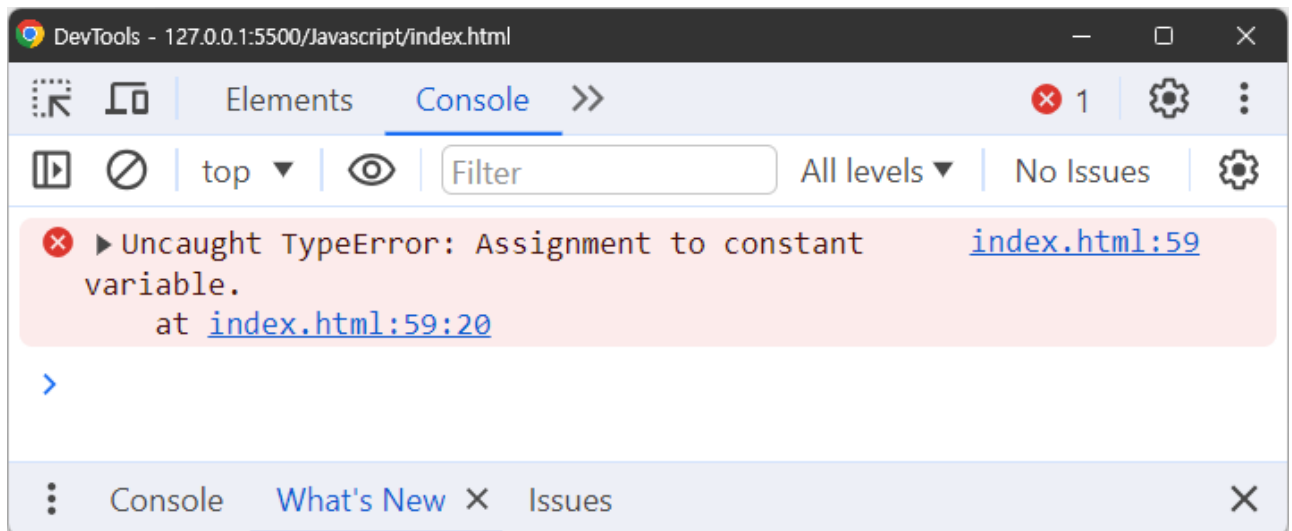
        constArray.push("추가" + i);
    }
    document.querySelector("#changeConstVar").innerHTML = constArray;
    // 재 할당 - 오류 발생
    constArray = ["재할당1", "재할당2"] ;
}
</script>

```



수행 결과

- 8 라인 : const 키워드로 constArray 변수 선언 및 초기화 설정
- 11 라인 : constArray 변수 값 추가
 - 재 할당이 아닌 값 변경은 가능 합니다. const 키워드는 재할당이 되지 않는 것 입니다.
- 15 라인 : constArray 변수에 값 재 할당
 - const 키워드는 재할당이 되지 않는 것으로 오류 발생합니다.



4. 변수 선언시 지켜야 할 것

- **var** 키워드는 사용하지 말자
 - 단 전역 변수가 필요할 때 최상위 수준에서 사용합니다.
- **const**를 가장 많이 사용 한다.
 - 변수값이 변경되는 경우 let를 사용한다.

```

let price = 10;
let totalPrice = 10 * 10

```

```
// .... 많은 라인이 있음
return totalPrice ; // 많은 라인에서 totalPrice 변경 가능성이 있음

// 만약 totalPrice 값이 한번 설정되고 변경이 되지 않으면
// const가 좋은 선택
const price = 10;
const totalPrice = 10 * 10
// .... 많은 라인이 있음
return totalPrice ; // 많은 라인에서 totalPrice를 변경 하지 못함
```

- 변수는 블록 유효 범위 내부로 격리하여 사용하자

- 함수를 만들어 사용 하면 변수를 함수안에 격리 하고
- for, if 블록은 내부에서 사용 되는 변수를 만들어 정보를 격리 하자
- 단일 책임의 원칙을 기억하자
- 함수 분리 리팩토링을 기억하자

5. 전역 변수를 사용하지 않는 방법

5-1. 즉시 실행 함수를 사용하자

모든 코드를 즉시 실행함수로 감싸면 모든 변수는 즉시 실행 함수의 지역변수가 됩니다.

```
> (function() {
  var v = 10; // 즉시 실행 함수의 지역 변수
  console.log("즉시 실행 함수 내부 v : " + v );
})();

console.log("즉시 실행 함수 외부 v : " + v );
즉시 실행 함수 내부 v : 10
```

✖ ▶ Uncaught ReferenceError: v is not defined
at <anonymous>:6:35

즉시 실행 함수는 즉시 실행되고 그 후 전역에서 바로 사라진다

```
(function () {
  var MYAPP = {};
  MYAPP.student = {
    name: '홍당무',
    gender: '남자'
  };
  console.log(MYAPP.student.name);
})();
```

5-2. 네임 스페이스

애플리케이션에서 전역변수 사용을 위해 다음과 같이 전역변수 객체 하나를 만들어 사용하는 것이다. (더글라스 크락포드의 제안)

전역에 네임스페이스 역할을 담당할 객체를 만들어 전역 변수로 사용할 객체를 따로 추가합니다.

```
> var myName = {}; // 전역 네임스페이스 객체
myName.name = '김길동';
console.log("이름 : " + myName.name); // 김길동

myName.person = {
  name: '홍길동',
  address: '서울'
};

console.log("person : " + myName.person.name); // 홍길동
이름 : 김길동
person : 홍길동
```

```
var MyApp = {};
MyApp.Name = {
  firstName: "홍",
  secondName: "당무"
}

MyApp.Print = function() {
  console.log("MyApp :: \n", MyApp) ;
  // MyApp ::
  //   { Name: { firstName: '홍', secondName: '당무' },
  //     Print: [Function] }
  console.log("Name :: ", MyApp.Name.firstName + MyApp.Name.secondName) ;
  // Name ::  홍당무
}

MyApp.Print();
```

5-3. 모듈 패턴

자바스크립트는 접근 제한자가 없어, 정보 은닉을 한정적으로 구현하기 위해 모듈 패턴을 사용합니다.

```
> var Counter = (function() {  
  //private 변수  
  var num = 0;  
  
  // 외부로 공개할 데이터나 메서드를 프로퍼티로 추가한 객체를 반환  
  return {  
    increase() {  
      return ++num;  
    },  
    decrease() {  
      return --num;  
    }  
  };  
})();  
  
// private 변수는 외부로 노출되지 않는다.  
console.log(Counter.num); //undefined  
  
console.log(Counter.increase()); // 1  
console.log(Counter.increase()); // 2  
console.log(Counter.increase()); // 3  
console.log(Counter.decrease()); // 2  
console.log(Counter.decrease()); // 1  
  
undefined  
1  
2  
3  
2  
1
```

\